ICS 381: Principles of AI

PA 3 – Programming Instructions

General Helpful Tips:

- Do not copy others' work. You can discuss general approaches with students, but do not share specific coding solutions.

- **NOTE1:** For this homework, we will be using numpy, sklearn, pandas, seaborn, matplotlib, and pgmpy.
- **NOTE2:** Be sure to implement the functions as specified in this document. In addition, you can implement any extra helper functions as you see fit with whatever names you like.

- Submit the required files only: [**system_crash_bn.py, machine_learning.py**]

Autograder Note: the autograder on gradescope will take around **90 seconds** to complete.

NOTE: You ***may*** get error when importing pgmpy saying there is no google and xgboost module. So, you may need to install

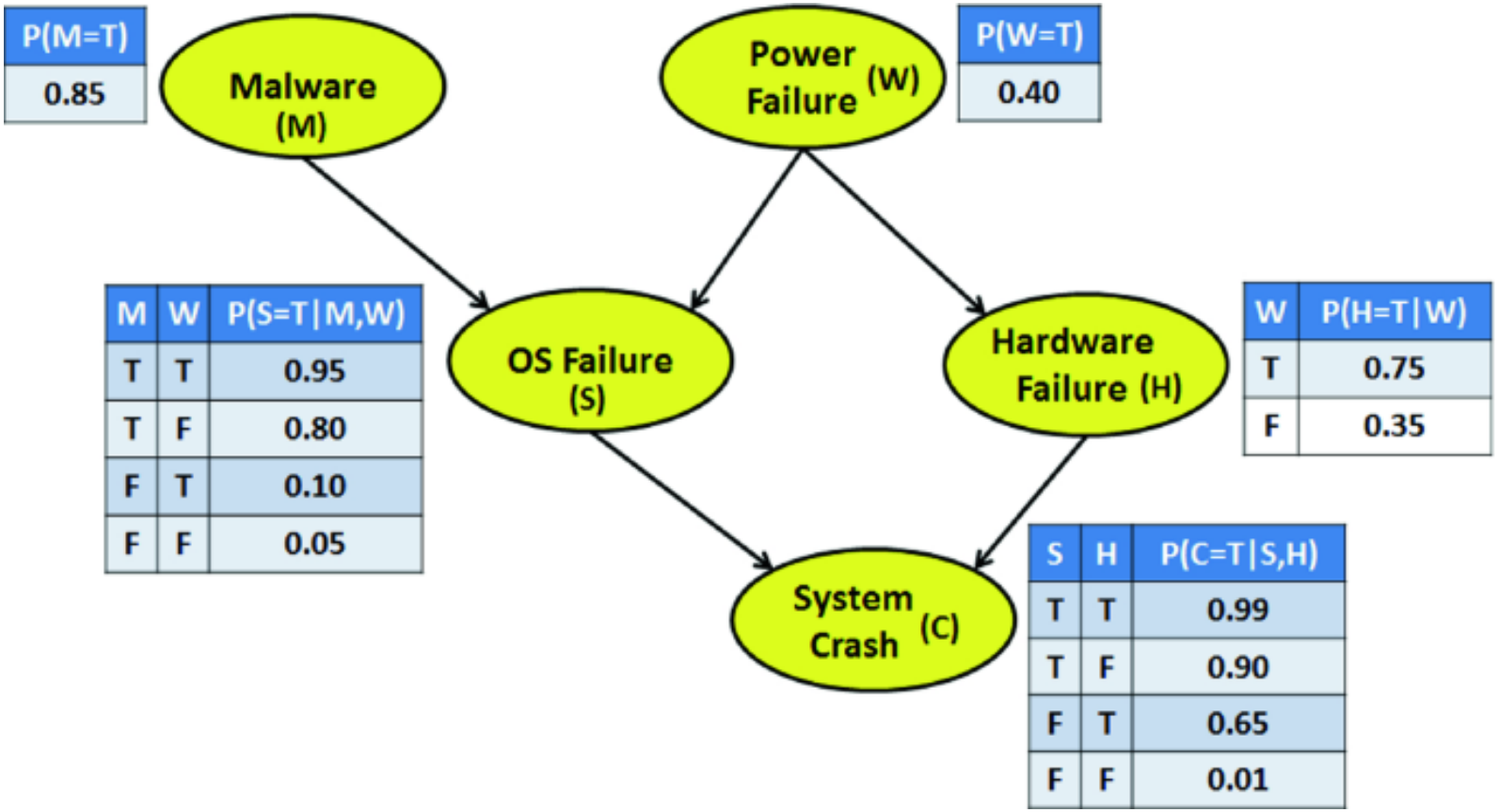
```
conda install conda-forge::google-generativeai
conda install conda-forge::xgboost
```

and you ***might*** need to update pgmpy to v1.0.0 if you get DiscreteBayesianNetwork import error, so do:

```
conda install conda-forge::pgmpy
conda install conda-forge::pyro-ppl
```

Bayesian Networks

For this homework, you will implement the simple Bayesian network in the figure below using pgmpy package (bn from link).



P(M=T)
0.85

P(W=T)
0.40

M	W	P(S=T|M,W)
T	T	0.95
T	F	0.80
F	T	0.10
F	F	0.05

W	P(H=T|W)
T	0.75
F	0.35

S	H	P(C=T|S,H)
T	T	0.99
T	F	0.90
F	T	0.65
F	F	0.01

system_crash_bn.py: Implement system crash BN

Function Name: `generate_bn()`
Returns: `sc_bn`

This function returns a bayesian network for the system crash BN in the figure above. Be sure to do:

- It is recommended that you see alarm_bn.py code for how the Alarm network is implemented using pgmpy. You can find this code on blackboard.
- Use variable names M, W, S, H, and C.
- The variables are binary values True and False. Be sure to set these in state_names.
- Take care with setting up the cpd table values matrix.

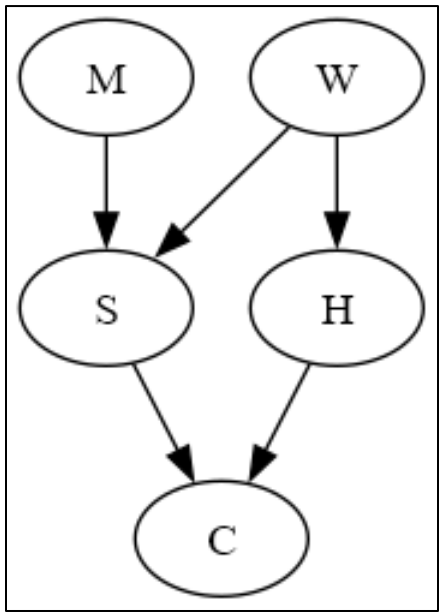
Function Name: `exact_inference(sc_bn, variables, evidence=None)`
Returns: `query_probability`

This function runs exact inference for the query $P(variables \mid evidence)$ for the system crash BN. Use variable elimination algorithm to do this. Variables is a list of variable strings, and evidence is a variable-value dictionary.

Test your code on test_bn.py

You can test your bn code by running test_bn.py. The following are my implementation results:

```
P(C) = +----------+----------+
| C        |   phi(C) |
+==========+==========+
| C(True)  |   0.7812 |
+----------+----------+
| C(False) |   0.2188 |
+----------+----------+
P(H | -s, -c) = +----------+----------+
| H        |   phi(H) |
+==========+==========+
| H(True)  |   0.2314 |
+----------+----------+
| H(False) |   0.7686 |
+----------+----------+
P(W | c) = +----------+----------+
| W        |   phi(W) |
+==========+==========+
| W(True)  |   0.4520 |
+----------+----------+
| W(False) |   0.5480 |
+----------+----------+
P(C | -s, h) = +----------+----------+
| C        |   phi(C) |
+==========+==========+
| C(True)  |   0.6500 |
+----------+----------+
| C(False) |   0.3500 |
+----------+----------+
P(S, C | -m) = +----------+----------+-----
| S        | C        |   phi(S,C) |
+==========+==========+============+
| S(True)  | C(True)  |     0.0666 |
+----------+----------+------------+
| S(True)  | C(False) |     0.0034 |
+----------+----------+------------+
| S(False) | C(True)  |     0.3098 |
+----------+----------+------------+
| S(False) | C(False) |     0.6202 |
+----------+----------+------------+
```



Machine Learning

For this homework, you will various model selection and evaluation functions that can be used for a wide range of binary classification datasets.

machine_learning.py: Implement data preprocessing

Function Name: `preprocess_dataset(train_df, test_df, discrete_columns, continuous_columns, label_column)`
Returns: `X_train, y_train, X_test, y_test`

Note: you can make a numpy array of columns of pandas dataframe as follows df[columns].values

Implementation: train_df and test_df are pandas dataframes for training and test sets, respectively. They contain discrete, continuous features, and the binary label. The discrete feature columns are given by the list discrete_columns. The continuous features are the list continuous_columns. The label is the label_column. Do the following:

1. Construct a numpy array of the discrete features for train_df and test_df. You should have two numpy arrays X_train_disc and X_test_disc.
2. Construct a numpy array of the continuous features for train_df and test_df. You should have two numpy arrays X_train_cont and X_test_cont.
3. Fit a one-hot encoder on the discrete features of the training set. Then use this encoder to transform the discrete features of training and test sets.
4. Fit a standard scale on the continuous features of the training set. Then use this scaler to transform the continuous features of training and test sets.
5. Construct X_train by hstacking the two processed training feature matrices above. Similarly, do the same to construct X_test. Note that you should stack with ordering [one-hot features, continuous features].
6. Setup y_train and y_test. These should be 1D numpy arrays.
7. Return X_train, y_train, X_test, y_test

Note: your function should work for any binary classification dataset for any number of features. The autograder will test your code against randomized binary classification datasets.

machine_learning.py: Implement model selection & evaluation

Here we are going to implement model selection and evaluation functions using K-fold CV. This function will involve: splitting a dataset into k-folds, looping k times to do the CV iterations, in each iteration you train and evaluate models. You will gather the results in a dictionary.

Function Name: `k_fold_cv(data_df, discrete_columns, continuous_columns, label_column, k=5)`
Returns: `cv_results_df`

data_df is a pandas dataframe for a binary classification dataset containing features and label. We want to apply K-fold CV on this dataset. Perform the following steps:

1. Using StratifiedKFold with shuffle=True and random_state=834307, do the CV loop on (train_index, test_index). Note that you can generate the indices using skf.split(data_df.values, data_df[label_column].values).
2. Setup a list for results = [ ]
3. In each CV loop, do:
    a. Construct train_df and test_df from data_df using the indices.
    b. Get X_train, y_train, X_test, y_test using `preprocess_dataset`
    c. Train and evaluate the models in the table below. For each model, fit on (X_train, y_train), then compute metrics on (X_test, y_test). The metrics are ROC-AUC and PR-AUC. Note for these metrics you should get class-1 probability predictions using model.predict_proba(X)[:,1]
    d. Append these results using results.append([model_name, fold_index, roc_auc_test, pr_auc_test]) where model_name is as shown in the table and fold_index corresponds to loop folds 1, 2, …, K.
4. After finishing the loop, construct the `cv_results_df` from results list. **Use columns** ['model name', 'fold id', 'roc-auc', 'pr-auc']
5. Return the `cv_results_df` dataframe.

Note: make sure you follow these steps. Check that your `cv_results_df` has the same output, column names, and values as in `test_ml_output.txt` when you run `test_ml.py`.

Model name	sklearn class	Argument setting
logreg1	LogisticRegression	penalty='l1', solver='liblinear'
logreg2	LogisticRegression	penalty='l2'
svmrbf	SVC	kernel='rbf', probability=True, random_state=834307
svmpoly	SVC	kernel='poly', degree=4, probability=True, random_state=834307
3nn	KNeighborsClassifier	n_neighbors=3
5nn	KNeighborsClassifier	n_neighbors=5
ab50	AdaBoostClassifier	n_estimators=50, algorithm='SAMME', random_state=834307
ab150	AdaBoostClassifier	n_estimators=150, algorithm='SAMME', random_state=834307
rf50	RandomForestClassifier	n_estimators=50, random_state=834307
rf150	RandomForestClassifier	n_estimators=150, random_state=834307

Note: it is helpful to setup this table as a global dictionary {modelname: sklearnclass}. This way you can use this dictionary in your functions to grab and setup the relevant sklearn class.

Function Name: `determine_best_model(results_df, metric_col)`
Returns: `best_model_name`

Using results_df of a K-fold CV, determine the best model based on average fold metric_col performance.
Note that your function should work for any results_df and metric_col.
You can assume that this df has columns ['model name', 'fold id', metric_col].
Implement this however you want. However, you can utilize panda's groupby to get this done in very few lines.

Function Name: `evaluate_best_model(best_model_name, train_df, test_df, discrete_columns, continuous_columns, label_column)`
Returns: `roc_auc_test, pr_auc_test`

Now we want to train the best model on the entire training set and evaluate it on a held-out test set. The entire training set will be passed by the user as train_df and test set as test_df. So, for this function do the following:

1. Get X_train, y_train, X_test, y_test using `preprocess_dataset`
2. Train the best-model on the training set.
3. Evaluate pr-auc and roc-auc on the held-out test set.
4. Return the test roc-auc score, and test pr-auc score.

Test your code on test_ml.py

You can test your machine learning code by running test_ml.py. The code operates on the stroke classification dataset in 'stroke.csv'. The continuous features are: age, avg_glucose_level, and bmi. The rest are discrete features. The label stroke is binary {0, 1}.

My implementation output can be found in test_ml_output.txt.